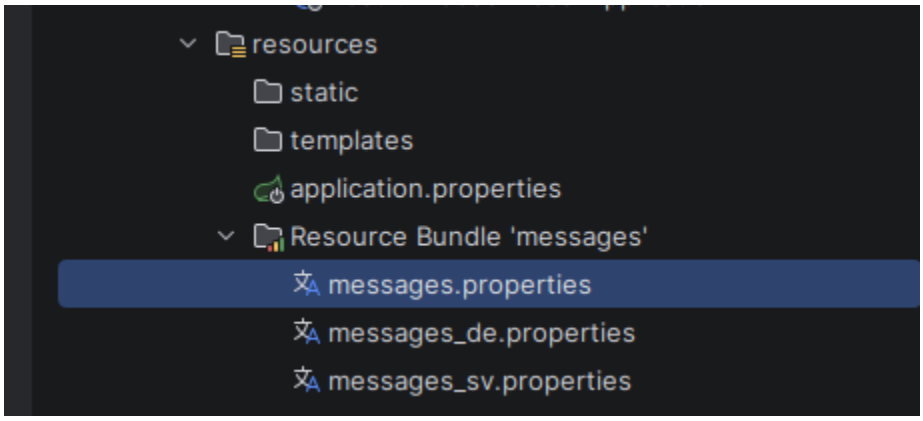
1)

A) Add support for Internationalization in your application allowing messages to be shown in English, German and Swedish, keeping English as default.

B) Create a GET request which takes "username" as param and shows a localized message "Hello Username". (Use parameters in message properties)



Created three Files– message.properties, message_de.properties, message_sv.properties

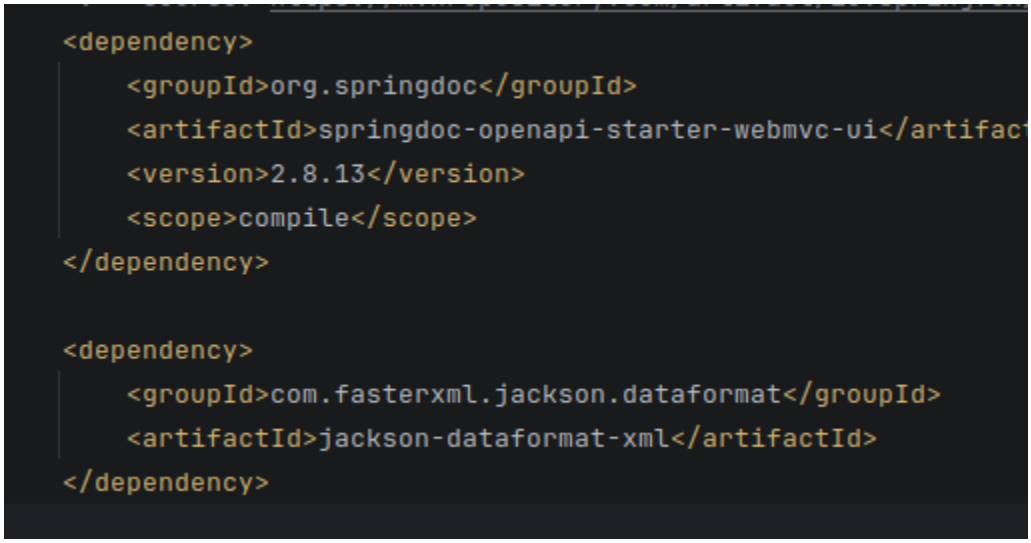
```
@Autowired
private MessageSource messageSource;

@GetMapping("/hello/in")
public String helloUser(@RequestParam String username, Locale locale){
    return messageSource.getMessage( code: "hello.user",new Object[] {username}, locale);
}
```

2)Content Negotiation

A) Create POST Method to create user details which can accept XML for user creation.

B) Create GET Method to fetch the list of users in XML format.



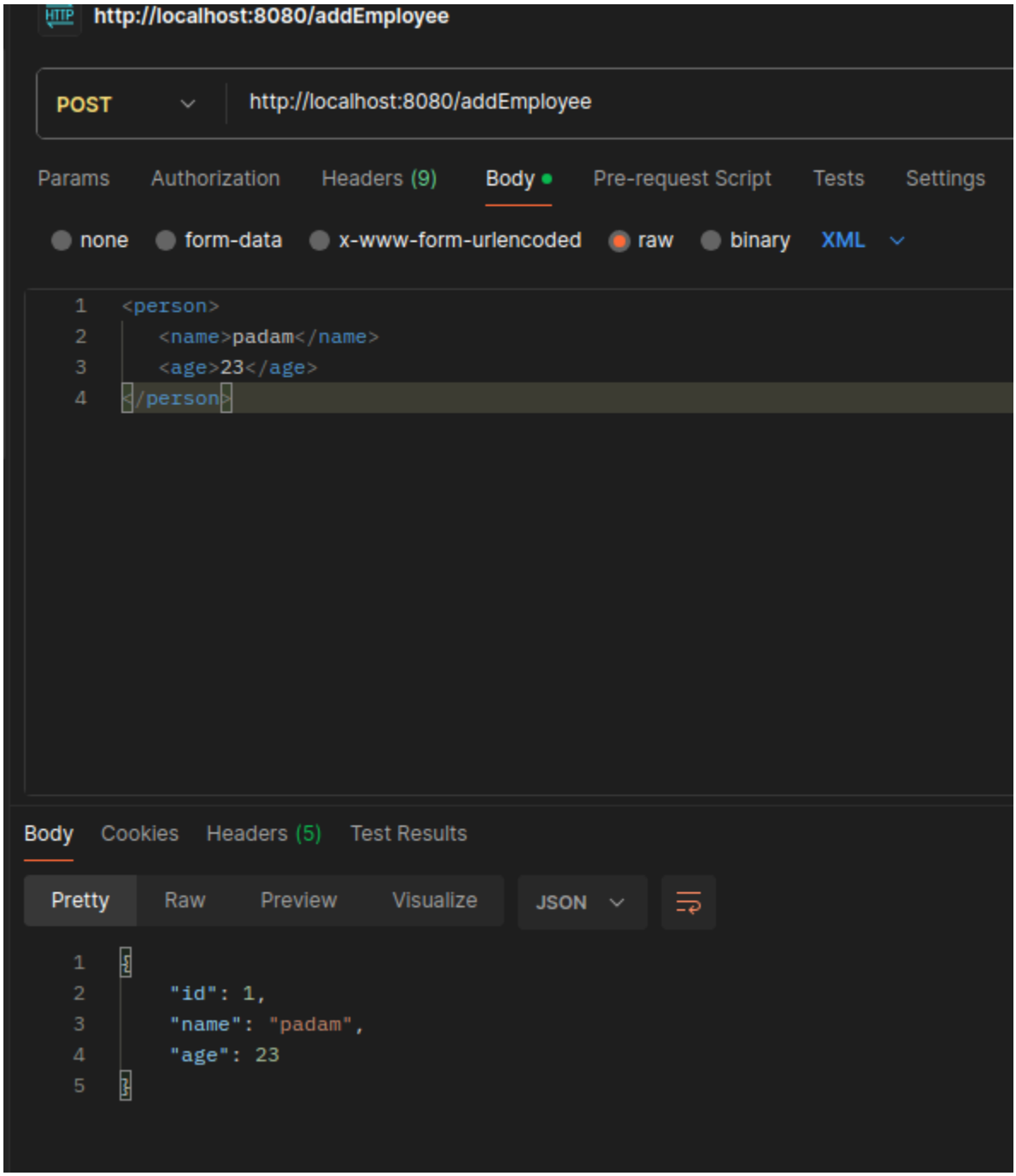
```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifac
    <version>2.8.13</version>
    <scope>compile</scope>
</dependency>

<dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-xml</artifactId>
</dependency>
```

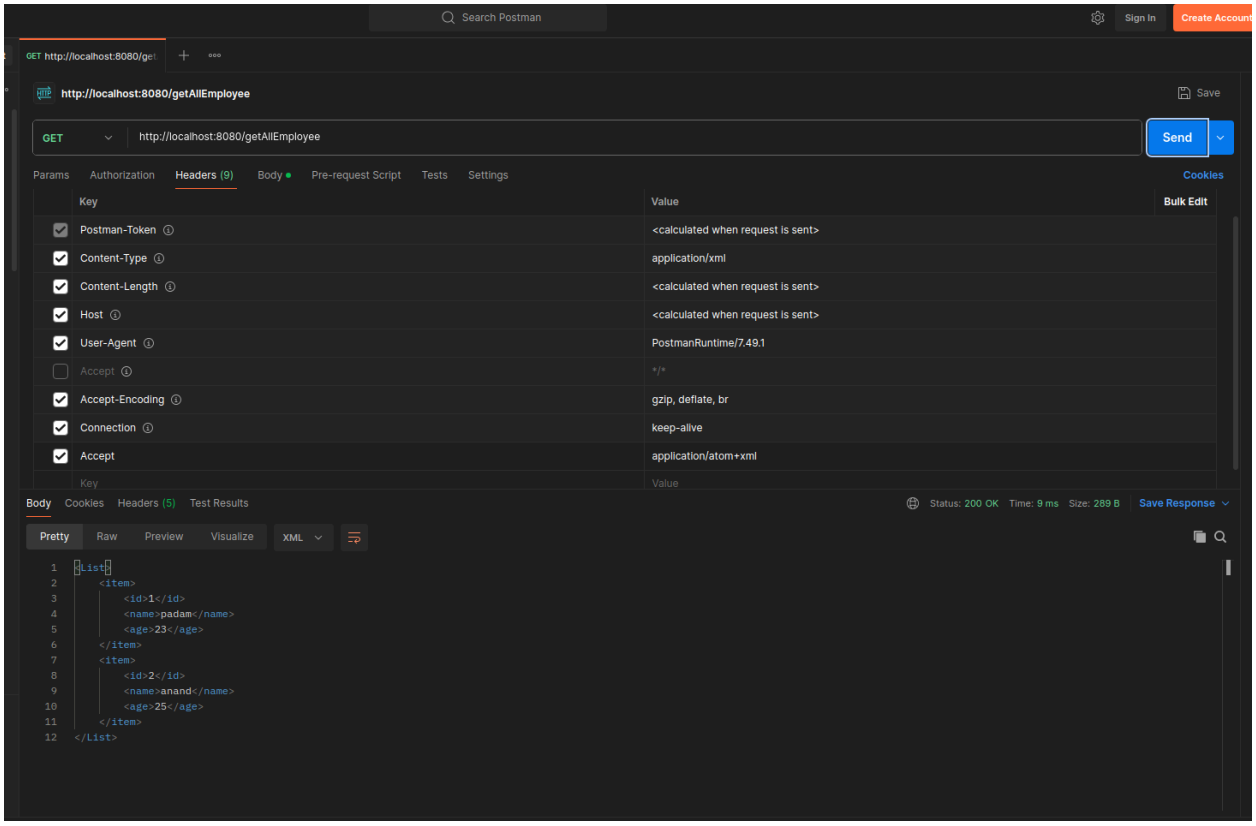
I added that jackson -dataformat-xml Dependency

So my controller’s mapping work for both xml and json now based on wat user send and user want it work for both .

A) Added object using xml format -
Means sending data in xml format and it accepting xml ad saving in database


http://localhost:8080/addEmployee
POST
http://localhost:8080/addEmployee
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
none
form-data
x-www-form-urlencoded
raw
binary
XML
1 <person>
2 <name>padam</name>
3 <age>23</age>
4 </person>
Body
Cookies
Headers (5)
Test Results
Pretty
Raw
Preview
Visualize
JSON
1 {
2 "id": 1,
3 "name": "padam",
4 "age": 23
5 }

B) Accepting data in xml format with adding(accept : application/atom+xml)

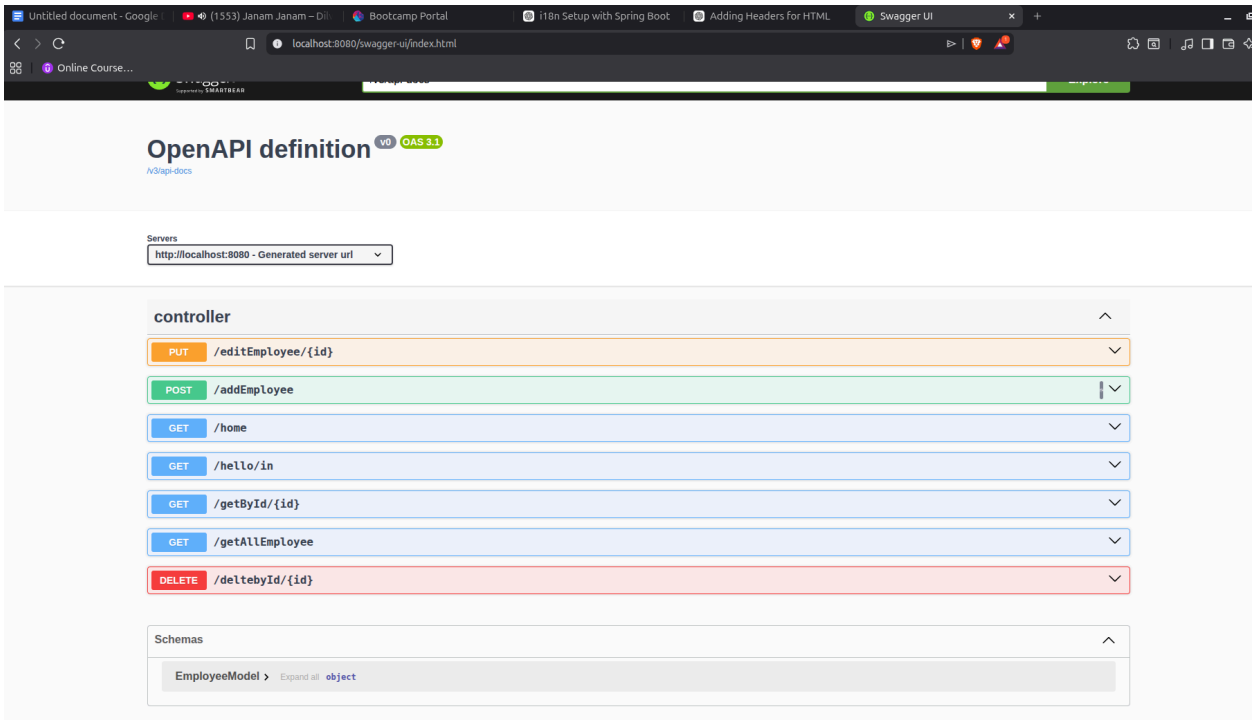


3) Swagger

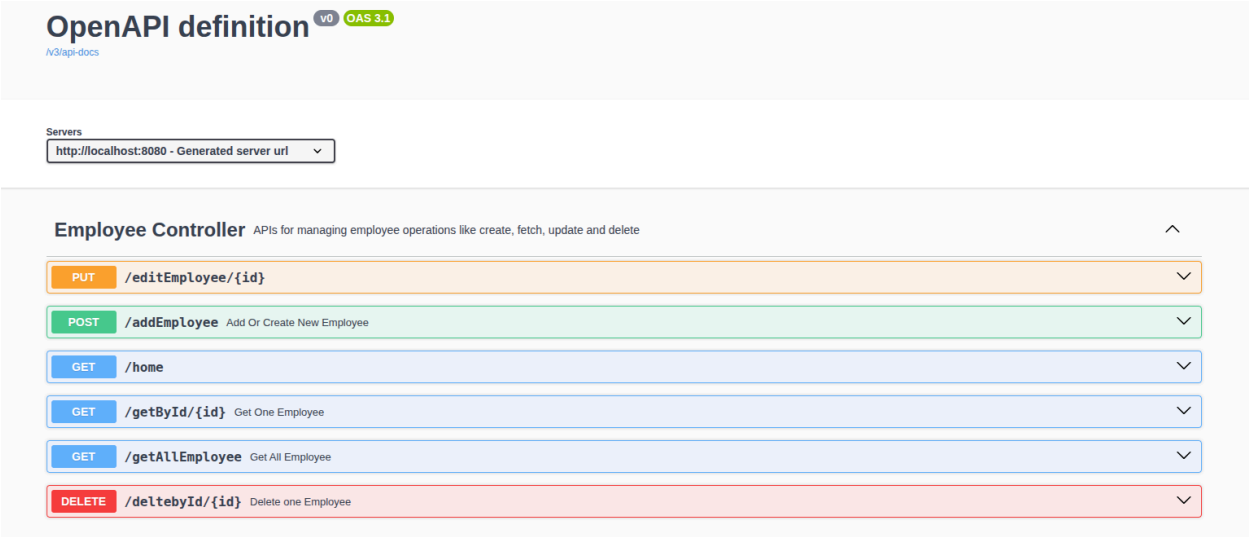
A) Configure the swagger plugin and create a document of the following methods: Get details of the User using GET request. Save details of the user using POST request. Delete a user using DELETE request.

B) In swagger documentation, add the description of each class and URI so that in swagger UI the purpose of class and URI is clear.

A) Configured Swagger plugin and create documentation.



b) Added the description of each class and URI



Static and Dynamic filtering

A) Create API which saves details of User (along with the password) but on successfully saving returns only non-critical data. (Use static filtering)

B) Create another API that does the same by using Dynamic Filtering.

A) API Created

```
package com.ttn.restfulwebservices1.controller;

import com.ttn.restfulwebservices1.model.EmployeeModel;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class StaticFilterQ4 {

    @PostMapping("/employee/static")
    public EmployeeModel saveEmployee(@RequestBody EmployeeModel employee){
        return employee;
    }

}
```

Added @JsonIgnore added to static filter Password .

```java
@Schema(description = "Employee entity representing employee details")
public class EmployeeModel {
    @Id
    @GeneratedValue(strategy =  GenerationType.IDENTITY)
    @Schema(description = "Unique Employee Id")
    Long id;
    @NotBlank(message = "Name is mandatory")
    @Size(min = 2, message = "Name must have at least 2 characters")
    @Schema(description = "Employee name")
    String name;

    @NotNull(message = "Age is mandatory")
    @Min(value = 18, message = "Age must be at least 18")
    @Max(value = 60, message = "Age must not exceed 60")
    Integer age;

    @JsonIgnore
    String password;
}
```

B) Dynamic Filtering using views applied

```java
@GetMapping("/users/{id}")
@JsonView(Views.Public.class)
public EmployeeModel getEmployee(@PathVariable Long id){
    return service.getEmployeeById(id);
}

@GetMapping("/users/internal/{id}")
@JsonView(Views.Internal.class)
public EmployeeModel getEmployeeInternal(@PathVariable Long id){
    return service.getEmployeeById(id);
}
```

5) Versioning Restful APIs

Create 2 API for showing user details. The first api should return only basic details of the user and the other API should return more/enhanced details of the user,

Now apply versioning using the following methods:

A) MimeType Versioning

B) Request Parameter versioning

C) URI versioning

D) Custom Header Versioning

URI versioning

```
private ModelRepo3 repo3;

//URI Versioning
@GetMapping("/v1/user/{id}")
public Model2 getUserV1(@PathVariable Long id) {
    return repo2.findById(id).orElseThrow(()-> new ResourceNotFoundException("Employee Not Exsist With Id"));
}

@GetMapping("/v2/user/{id}")
public Model3 getUserV2(@PathVariable Long id) {
    return repo3.findById(id).orElseThrow(() -> new ResourceNotFoundException("Not Exsist with id"));
}
```

Request Parameter versioning

```
//Request Parameter Versioning
@GetMapping(value="/user/{id}", params="version=1")
public Model2 getUserParamV1(@PathVariable long id) {
    return repo2.findById(id).orElseThrow(()-> new ResourceNotFoundException("Employee Not Exsist With Id"));
}

@GetMapping(value="/user/{id}", params="version=2")
public Model3 getUserParamV2(@PathVariable Long id) {
    return repo3.findById(id).orElseThrow(() -> new ResourceNotFoundException("Not Exsist with id"));
}
```

Custom Header Versioning

```
}

@GetMapping(value="/user/header/{id}", headers="X-API-VERSION=1")
public Model2 getUserHeaderV1(@PathVariable Long id) {
    return repo2.findById(id).orElseThrow(()-> new ResourceNotFoundException("Employee Not Exsist With Id"));
}

@GetMapping(value="/user/header/{id}", headers="X-API-VERSION=2")
public Model3 getUserHeaderV2(@PathVariable Long id) {
    return repo3.findById(id).orElseThrow(() -> new ResourceNotFoundException("Not Exsist with id"));
}
```

MIME type versioning

```
//MIME type versioning

@GetMapping(
        value="/user/produces/{id}",
        produces="application/vnd.company.app-v1+json"
)
public Model2 getUserMimeV1(@PathVariable Long id) {
    return repo3.findById(id).orElseThrow(() -> new ResourceNotFoundException("Not Exsist with id"));
}

Change signature
@GetMapping(
        value="/user/produces/{id}",
        produces="application/vnd.company.app-v2+json"
)
public Model3 getUserMimeV2(@PathVariable Long id) {
    return repo3.findById(id).orElseThrow(() -> new ResourceNotFoundException("Not Exsist with id"));
}
```

HATEOAS

A) Configure hateoas with your springboot application. Create an api which returns User Details along with url to show all topics.

Added url of get All Employees

```
@PostMapping("/addEmployee")
@Operation(summary = "Add Or Create New Employee" , description = "Create or Add new Employee record in new database")
public EntityModel<EmployeeModel> addEmp(@Valid @RequestBody EmployeeModel emp){
    EmployeeModel employeeSaved= service.saveEmployee(emp);

    // wrap user
    EntityModel<EmployeeModel> resource = EntityModel.of(emp);

    // add link to all users API
    resource.add(linkTo(methodOn(Controller.class)
            .getAllEmp()).withRel("all-users"));
    return resource;

}
```